

One audit run, start to finish

Every file, function, database write, and LLM call the platform executes between registering an AI system and reading back its governance report — in execution order, with the response each stage returns to its caller.

LAYERS 5 + registry	SPECIALIST AGENTS 7	COUNCIL AGENTS 3	HASH CHAINS 2 + content digest
TRACED FROM backend/app			

EXECUTION ORDER

- 00 Orientation & map
- 01 App registry
- 02 Run creation & kickoff
- 03 Context assembly
- 04 Adaptive orchestrator
- 05 Human approval gate
- 06 Metric execution
- 07 Specialist agents
- 08 Deliberation council
- 09 Action & reporting
- 10 Finalize

CROSS-CUTTING

- LLM gateway & call logs
- Integrity chains
- Failure & resume
- Read-back API surface
- Everything a run writes

00 Orientation & map

The platform is a FastAPI backend (`backend/app`) with a Vite/React frontend. An audit is one `EvaluationRun` row driven through eight phases by a single background job. Everything a phase learns is written append-only before the next phase reads it, so a run is reconstructable from the database alone.

THE TWO ENTRY POINTS

`backend/app/main.py` → `create_app()` mounts `api/router.py` under `/api/v1`, installs the exception handlers, CORS, and a bearer-token middleware that protects `POST/PUT/PATCH/DELETE` only (GET stays open so the SSE progress stream and `<img>` media routes work without headers). On startup it also spawns a daemon thread running `reconcile_interrupted_runs()` — see § Failure & resume.

HTTP REQUEST THREAD



BackgroundTask

BACKGROUND JOB • advisory lock per ai_system_id



pauses at status=planned → POST /approve-plan resumes at metric execution

human

PERSISTENCE — every phase writes before the next one reads



LLM GATEWAY — wraps every model call in all four LLM-bearing phases



The orchestrate request returns 202 after one synchronous status bump; the pipeline itself runs off the request thread under a per-system advisory lock. Only the approval gate leaves the process — it parks the run and waits for a person.

PHASE & STATUS VOCABULARY

Two enums in `models/enums.py` track progress. `RunPhase` is the ordered pipeline position; `RunStatus` is the lifecycle state (and what clients poll for a terminal value).

RUNPHASE	RUNSTATUS WHILE IN IT	OWNER
created	created	route handler
context_assembly	context_assembly	context_assembly/assembler.py
adaptive_orchestrator	planned	adaptive_orchestrator/planner.py
metric_execution	metrics_running	specialist_agents/metric_execution.py
specialist_agents	agents_running	specialist_agents/agent_execution.py
deliberation_council	council_running	deliberation_council/deliberation.py
action_reporting	unchanged (non-terminal)	action_reporting/reports.py
completed	completed / degraded / failed	orchestration.py

degraded is half-terminal: metric and agent execution can set it mid-pipeline, so it is not proof the run finished. A success finalize must not overwrite it; a failure finalize must be able to. That rule lives in `_finalize_run()`.

01 App registry — registering the system under audit

Nothing can be audited until it exists in the registry. Registration turns one nested payload into ~16 normalized fact tables in a single transaction, derives a preliminary risk tier, and — critically for the engine — creates one

`AISystemCapability` per endpoint. Those capability rows are what every later phase treats as the audit surface.

POST /api/v1/ai-systems/register

```
└ api/routes/ai_systems.py :: register_ai_system()
  └ services/registration.py :: register_ai_system(session, payload)
    ├── _validate_catalogs()           # every enum-ish field vs registration_config catalogs
    ├── _canonical_framework_ids()     # configs/frameworks/registry.py :: get_framework_knowledge()
    ├── _first_duplicate() x4          # reject dup model/endpoint/framework/control BEFORE insert
    ├── _require_registered_fields()   # skipped when status == "draft"
    ├── risk_classification_service.py :: screen_risk(answers)
    ├── _compute_completeness()        # 15-item recommended-profile checklist → %
    ├── session.add(AISystem) → session.flush() # assigns id, no commit
    ├── _build_usage_context()          → ai_system_usage_contexts
    ├── _build_context_profile()        → application_context_profiles # 5 seeded sections
    ├── AISystemOwner xN, _model_row() xN → flush # model ids for endpoint wiring
    ├── _endpoint_row() xN, AISystemFramework xN, AISystemRiskScreening
    ├── _data_source_row() / _rag_row() / _agent_row() / _dependency_row() / _document_row()
    ├── AISystemSecurityControlStatus xN
    ├── _build_capability_for_endpoint() xN # ONLY when not draft ← the engine reads these
    └ session.commit()
      ├── IntegrityError → rollback + _integrity_error_conflict() → 409 naming the REAL table
      └ ApplicationError → rollback + re-raise (422)

→ _build_registration_read() # re-queries all 14 child tables
```

INPUT

`AISystemRegistrationCreate` — `system`, `owners[]`,
`models[]`, `endpoints[]`, `frameworks[]`,
`usage_context`, `risk_screening`, plus optional
`data_sources`, `rag_configuration`,
`agent_configuration`, `security_posture`,
`dependencies`, `documents`.

RESPONSE • 201

`AISystemRegistrationRead` — the whole nested record
back, plus `preliminary_risk_score`,
`preliminary_risk_tier`, `triggered_risk_factors[]`,
`profile_completeness`,
`missing_recommended_fields[]`, `registration_status`.

The four engine-facing fields

Most registration data is documentation. Four fields on `AISystem` are load-bearing for the pipeline:

FIELD	SET FROM	READ BY
<code>risk_tier</code>	<code>clamp_to_risk_tier(screening...)</code>	Planner budget bonus; <code>_verify_even_when_passing()</code> — <code>high</code> forces agents to probe even when metrics pass
<code>selected_frameworks</code>	canonical framework ids	Regulatory ingester; metric-plan mapping filter
<code>target_endpoint_ref</code>	first endpoint's URL	<code>get_target_model_client_for_system()</code> adapter inference
<code>modality</code>	payload	Metric applicability, unioned with each capability's own modality

CAPABILITIES, AND HOW THEY GET RICHER

`_build_capability_for_endpoint()` creates a minimal capability (type `inference`, modality defaulting to text, no input schema). Two routes enrich them, and both matter downstream:

- `POST /ai-systems/{id}/capabilities/import-from-catalog` → `services/capability_import.py` fetches `GET {target_endpoint_ref}/catalog` and creates one capability per endpoint *with its real* `input_schema` *and modality*.
- `PATCH /ai-systems/{id}/capabilities/{cid}` for hand edits.

WHY THE SCHEMA MATTERS

`input_schema` and `modality` drive the two fail-closed gates in `_execute_probe_plan()`. Without them a probe against an image endpoint is skipped as text-only, and a probe against a document-consuming endpoint is skipped because the question would be parsed *as* the document. A capability with no schema silently reduces probe coverage — see § 07.

RETRIEVAL CONTEXT

`POST /ai-systems/{id}/retrieval-context[/upload]` ingests a corpus (PDF via `pypdf`, DOCX via `python-docx`, or pasted text — `_extract_document_text()`). RAGAS/DeepEval grounding metrics read it; without it they *skip with a reason* rather than fabricate a hallucination score (`missing_grounding_corpus_reason()`).

02 Run creation & kickoff

POST /api/v1/evaluation-runs

```
└─ services/evaluation_runs.py :: create_evaluation_run()
  └─ EvaluationRun(status=created, current_phase=created,
                    selected_frameworks[], selected_metrics[], selected_capabilities[])
    → 201 EvaluationRunRead
```

POST /api/v1/evaluation-runs/{run_id}/orchestrate

```
└─ api/routes/evaluation_runs.py :: run_governance_pipeline()
  └─ service.get_evaluation_run()
  └─ if phase == created: status=context_assembly, phase=context_assembly,
    │                       started_at=now → session.commit() # synchronous, so the
    │                                                                # Live Run view leaves "Run Setup" at once
  └─ background_tasks.add_task(orchestration.run_governance_pipeline_job, run_id, payload)
    → 202 {"run_id": "...", "status": "accepted"}
```

The UI's payload is fixed in `frontend/src/api/governanceApi.ts` → `orchestrateRun()`: `{mock_score: 0.5, evaluator_name: "auto", requested_by: "frontend", notes: "Triggered from UI.", require_plan_approval: true}`. Notably it sends *no* `logs` and *no* `force_metric_status` — so logs get synthesized and every metric reports its real score-vs-threshold outcome.

The background job

```
orchestration.py :: run_governance_pipeline_job(run_id, payload)
└─ Session(engine) → get_run_or_raise() → ai_system_id
└─ concurrency.ai_system_run_lock(engine, ai_system_id=...)
  │   # Postgres advisory lock on a DEDICATED connection (never the ORM session's –
  │   # a pooled connection can change between commits and orphan the lock).
  │   # A second run against a busy system fails fast instead of interleaving probes.
└─ Session(engine) → run_governance_pipeline(session, run_id, payload)
  │   ↳ returns None when paused for approval
└─ except → logger.exception + _finalize_run(status=failed, error=exc)
└─ else, if result is not None → _finalize_run(status=completed)
    # on a FRESH session: a poisoned pipeline session must never be able to
    # park a run at council_running forever
```

FIRST ACT OF THE PIPELINE

```
run_governance_pipeline(session, run_id, payload)
└─ run.pipeline_payload = payload.model_dump(mode="json") → commit
  │   # captured UNCONDITIONALLY so both the approval-resume and the crash-resume
  │   # paths replay the options actually requested, not defaults
└─ logs = payload.logs or synthesize_logs_for_system(ai_system)
  │   # context_assembly/log_synthesizer.py – deterministic, seeded from
  │   # ai_system.id % 997; category-aware; DELIBERATELY partial so real gaps surface
└─ _context_notes(notes, source, count) # stamps log provenance into the audit trail
```

03 Layer 1 — Context assembly

Fully deterministic: no LLM, no wall clock, no randomness. Same input always yields the same output, which is what lets the result be hash-chained and reproduced.

```
services/context_assembly/assembler.py :: assemble_context(session, run_id, payload)
├─ get_run_or_raise() → _ensure_can_assemble(run) # 409 INVALID_RUN_TRANSITION if terminal
├─ log_analyzer.py :: analyze_logs(entries)
│   └─ LogAnalysisSummary — key-sorted Counters:
│       total_requests, empty, request_category_counts, demographic_coverage,
│       jurisdiction_coverage, outcome_counts, modality_counts,
│       pii_request_count, flagged_request_count, observed_* lists
├─ regulatory_ingester.py :: ingest_regulations(session, selected_frameworks)
│   └─ SELECT FrameworkMapping WHERE framework_id IN (...) # DB-seeded controls
│   └─ configs/frameworks/registry.py :: get_framework_knowledge(id)
│       # eu_ai_act · iso_42001 · nist_ai_rmf · mitre_atlas ·
│       # owasp_llm_top_10 · owasp_agentic_ai — rubrics, probe templates,
│       # severity thresholds, citation format
│   └─ RegulatoryContextRead(frameworks[], resolved_frameworks[],
│       missing_frameworks[], control_count)
├─ coverage_gap_detector.py :: detect_coverage_gaps(log_analysis, regulatory_context)
│   └─ per CoverageRequirement: _evaluate_requirement()
│       demographic_coverage / jurisdiction_coverage / request_category /
│       outcome_balance → set-membership vs observed_*; pii_handling → count
│   └─ [CoverageGapRead(gap_id, dimension, severity, description,
│       expected[], observed[], control_refs[])]
├─ highest_gap_severity(gaps)
├─ _build_counts() → {log_requests, frameworks_resolved, frameworks_missing,
│   regulatory_controls, probe_templates, coverage_gaps}
├─ governance_state.py :: append_state_entry(entry_type="context_assembled",
│   source="context_assembly", phase=context_assembly)
│   # the FULL assembled context lives in the state payload → reconstructable
│   # from state alone, no process-local memory
├─ audit_ledger.py :: append_ledger_entry(event_type="context_assembly.completed",
│   actor_type=system, actor_id="context_assembly")
└─ run.status = context_assembly; result_summary["context_assembly"] = counts → commit
```

RETURNS

ContextAssemblyRead — state_sequence_number,
state_entry_hash, generated_at, log_analysis,
regulatory_context, coverage_gaps[], gap_count,
highest_gap_severity, counts.

READ BACK

GET /evaluation-runs/{id}/context-assembly →
get_latest_context() rebuilds the same object from the
highest-sequence context_assembled state entry. 404
if the phase never ran.

WHY LOGS ARE SYNTHESIZED

With no logs, `total_requests: 0` tripped the `empty` branch of every requirement, producing an identical "everything missing" gap set on every run — and since coverage gaps are the planner's only per-run variable, the planner then allocated an identical budget every time and looked static. The synthesizer gives each system its own partial, plausible sample instead.

04 Layer 2 — Adaptive orchestrator

Two-stage selection. A deterministic pass decides the candidate metric set from applicability filters; an LLM review pass may then *narrow* it — never widen it, and never by more than half.

```
services/adaptive_orchestrator/planner.py :: prepare_evaluation_plan(session, run_id, payload)
└ gateway.start_log_capture(run_id, "adaptive_orchestrator")
└ _apply_llm_plan_review(session, run_id) # stage 2 – see below
└ for entry in drain_log_capture(): session.add(LLMCallLog(run_id, **entry))
└ build_evaluation_plan(session, run_id) # stage 1 + assembly
└ plan.llm_review = <the review audit record>
└ append_state_entry(entry_type="evaluation_plan_prepared",
  | source="adaptive_orchestrator", phase=adaptive_orchestrator)
└ append_ledger_entry(event_type="evaluation_plan.prepared")
└ run.status = planned; result_summary["evaluation_plan"] = plan.counts → commit
```

Stage 1 — deterministic candidate set

```
build_evaluation_plan()
└ specialist_agents/metric_plans.py :: build_metric_plan(session, run_id)
  | └ SELECT AISystemCapability WHERE enabled → capability_types, capability_modalities
  | └ SELECT MetricConfig WHERE enabled ORDER BY metric_id
  | └ SELECT FrameworkMapping WHERE enabled ORDER BY control_ref
  | └ _filter_mappings_for_run() # keep mappings for run.selected_frameworks
  | └ _resolve_metric_ids() # explicit selection wins, else mapping-referenced set
  | └ _filter_metrics_for_run() # mapped OR (framework_match AND not default_seed)
  | └ _metric_applies_to_system() # applicable_capability_types n ; applicable_modalities n
  | | # (system modality UNION every capability modality –
  | | # the single system field would drop a real video endpoint)
  | └ _build_metric_plan_item() # joins each metric to its framework controls
  └ → MetricPlanRead(metric_count, control_count, metrics[])

└ _load_coverage_gaps() # from the Layer 1 state entry
└ group metrics by primary_agent → activated agents (sorted)
└ _assign_gaps_to_agents() # by dimension, then by control_ref → agent_names
└ weight[a] = METRIC_WEIGHT[metrics] + Σ SEVERITY_WEIGHT[gap] + RISK_TIER_BONUS[tier]
└ budget/base.py :: allocate_probe_budget(weights, total=PROBE_BUDGET_TOTAL=100)
└ _build_agent_item() # priority, probe_budget, assigned_metric_ids,
  | # target_dimensions/controls, coverage_gap_ids,
  | # instructions (prose), rationale
└ _build_priority_targets() # gaps rolled up per dimension, severity-sorted
└ _risk_rationale()
```

Stage 2 — the LLM review

LLM CALL #1

```
_apply_llm_plan_review(session, run_id)  # never raises – planning must survive no LLM
├ settings.adaptive_plan_review_enabled is False → {"reviewed": false, reason:...}
├ run.selected_metrics already set → {"reviewed": false, "explicit metric selection"}
├ no candidate metrics → {"reviewed": false, "no candidate metrics"}
├ get_governance_model_client() # Gateway-wrapped
├ client.complete(GovernanceModelRequest(
│   task="evaluation_plan_review",
│   prompt=_PLAN_REVIEW_PROMPT.format(system_name, system_type, risk_tier,
│   modality, description[:400], capability_types, context_summary,
│   metric_lines="CM-0xx | name | dimension" xN)))
├ _parse_plan_review(content) # first { ... last } → dict, else None
├ drops = [d for d in review["drop"] if d["metric_id"] in candidate_ids]
├ if len(drops) > 0.5 × |candidates| → applied=false, ignored_reason=... # safety cap
├ else run.selected_metrics = sorted(candidates - dropped) → flush
└ → audit {reviewed, candidate_count, dropped[], rationale, applied, kept_metric_ids}
```

MODEL RESPONSE SHAPE

```
{"drop": [{"metric_id": "CM-000", "reason": "..."}],
"rationale": "..."} — anything unparseable degrades to
{"reviewed": false} and the deterministic set stands.
```

RETURNS • 201

```
EvaluationPlanRead — risk_tier, metric_count,
coverage_gap_count, probe_budget_total: 100,
probe_budget_allocated, activated_agents[],
priority_targets[], risk_rationale, counts,
llm_review, plus the state hash.
```

Read back with `GET /{id}/evaluation-plan` (`get_latest_plan()`, rehydrated from the state entry) or `GET /{id}/metric-plan` (`build_metric_plan()`, recomputed live — this is what the approval modal renders).

05 The human approval gate

A real human-in-the-loop pause, implemented without a new enum value: the run simply stays at `RunStatus.planned` with `plan_approved_at IS NULL`. That pair is the single source of truth for "waiting on a reviewer" (`is_awaiting_plan_approval()`).

```
# still inside run_governance_pipeline()
run = get_run_or_raise(session, run_id)
if payload.require_plan_approval and run.plan_approved_at is None:
    _pause_for_approval()
    └─ append_ledger_entry(event_type="plan.awaiting_approval",
                          payload={phase: adaptive_orchestrator, message: ...})
    return None # the job sees None and does NOT finalize – run parks at 'planned'
```

POST /api/v1/evaluation-runs/{run_id}/approve-plan

```
└─ api/routes/evaluation_runs.py :: approve_plan()
  └─ current_user = Depends(get_current_user)
  └─ if current_user: payload.approved_by = current_user.name
    │   # the authenticated caller's name is authoritative – an approval
    │   # cannot be forged by supplying a different approved_by in the body
  └─ services/evaluation_runs.py :: approve_plan(session, run_id, payload)
    │   └─ not is_awaiting_plan_approval() → 409 PLAN_NOT_AWAITING_APPROVAL
    │   └─ selected_metrics == [] → 422 EMPTY_METRIC_SELECTION
    │   └─ selected_metrics is not None → _apply_manual_metric_selection()
    │     │   └─ run.selected_metrics = ids → commit
    │     │       → adaptive_orchestrator.prepare_evaluation_plan() # RE-PLANS:
    │     │           # new activated agents + probe budgets for the chosen subset,
    │     │           # and the LLM review is skipped (explicit selection)
    │   └─ plan_approved_at/by = now/user; status = metrics_running; phase = metric_execution
    │     └─ append_ledger_entry(event_type="plan.approved", actor_type=user,
    │                           payload={manual_selection, selected_metrics}, commit=False)
    │         # commit=False so status change + ledger land in ONE transaction
  └─ background_tasks.add_task(orchestration.resume_governance_pipeline_job, run_id)
    └─ → 200 EvaluationRunRead
```

```
orchestration.py :: resume_governance_pipeline(session, run_id)
└─ payload = GovernancePipelineRunCreate.model_validate(run.pipeline_payload)
  │   or GovernancePipelineRunCreate() # replays what was requested
└─ evaluation_plan = adaptive_orchestrator.get_latest_plan()
  └─ _execute_and_report(context_result=None, evaluation_plan=plan)
```

`reconcile_interrupted_runs()` deliberately skips these runs: a run waiting on a person is working exactly as designed, not orphaned.

06 Metric execution — the deterministic evidence layer

Twelve real evaluators score the planned metrics by probing the live target. Every metric produces exactly one `EvidenceRecord` and one `MetricResult`, committed the moment it is built — because the probe already happened whether or not the process survives to persist it.

```

services/specialist_agents/metric_execution.py :: run_metrics(session, run_id, payload)
└─ plan = build_metric_plan()
└─ evaluator = evaluators/registry.py :: get_evaluator(payload.evaluator_name)
    │   # "auto" (what the UI sends) → _AutoRoutingEvaluator
└─ target_client = model_clients/registry.py :: get_target_model_client_for_system(ai_system)
└─ run.status = metrics_running; phase = metric_execution → commit
    │   # up front, so live watchers see the layer activate before slow evaluators run
    │
└─ # worker snapshot on a THROWAWAY session – the commit above expired every ORM
    │ # attribute, and concurrent lazy refreshes from workers raise
    │ # "session is provisioning a new connection; concurrent operations not permitted"
    with Session(engine) as snapshot_session:
        │ worker_ai_system = snapshot.get(AISystem, id)
        │ resolved_capabilities = SELECT AISystemCapability ...
        │ resolved_endpoint_ref = evaluators/base.py :: resolve_evaluator_endpoint(
        │ │ │ ai_system, capabilities, run.selected_capabilities)
        │ target_modality.py :: register_capability_modalities(caps)
        │ │ # teaches the Gateway which endpoints are video, so its concurrency
        │ │ # cap can serialise renders – evaluators don't know they're doing it
        │
└─ gateway.start_log_capture(run_id, "metric_execution"); capture_buffer = get_log_buffer()
    │
└─ # resume-safety
    │ already_done = MetricResult WHERE run_id AND status IN (passed, failed) # never re-probe
    │ metrics = [m for m in plan.metrics if m.metric_id not in already_done]
    │ DELETE stale MetricResult+EvidenceRecord for metrics about to be retried
    │
└─ pool = ThreadPoolExecutor(max_workers=metric_execution_max_workers()) # default 3
    │ # bounded LOW: at 6 the shared Azure judge returned a 429 storm and backoff
    │ # inflated a run past 10 minutes
    │ for future in as_completed(futures, timeout=metric_execution_budget_for(n)):
    │ │ # _evaluate(metric), one short-lived Session per worker:
    │ │ └─ gateway.bind_log_capture(capture_buffer, agent_name=metric.tool_name)
    │ │ │ # contextvars don't cross threads; rebound per metric because
    │ │ │ # pool threads are reused
    │ │ └─ evaluator.evaluate(MetricEvaluationInput(metric, mock_score, force_status,
    │ │ │ │ source_name, session, ai_system, target_client,
    │ │ │ │ target_endpoint_ref, capabilities, run_id))
    │ │ └─ except → MetricEvaluationResult(status=error, payload={error_type, error})
    │ │ │ # one metric's blip must not discard every other metric's results
    │ except FuturesTimeout:
    │ │ for pending: future.add_done_callback(_record_orphaned_completion)
    │ │ │ # the straggler thread keeps running and may still be calling the target;
    │ │ │ # ledger event "metric.orphaned_after_timeout" records what it did
    │ pool.shutdown(wait=False) # never block on stragglers
    │ missing → _timed_out_result(reason="exceeded the metric-execution time budget") = skipped
    │
└─ for metric, evaluation in zip(metrics, evaluations, strict=True):
    │ EvidenceRecord(source_type, source_name=evaluation.tool_name, tool_name,
    │ │ raw_score, normalized_score, threshold, passed, payload) → flush
    │ MetricResult(metric_id, dimension, tool_name, status, scores,
    │ │ │ evidence_ids=[str(evidence.id)]) → commit per metric
    │
└─ run.status = degraded if any error else metrics_running
└─ run.result_summary = {metric_plan_count, evidence_created, metric_results_created,
    │ │ metrics_failed, metrics_reused_from_prior_attempt,

```

```
|         evaluator_name, mock_execution}
└─ for entry in drain_log_capture(): session.add(LLMCallLog(...)) → commit
```

The evaluator registry

`services/evaluators/registry.py` holds twelve real evaluators. The `"auto"` router dispatches each metric to the evaluator its own `MetricConfig.tool_name` names:

EVALUATOR	WHAT IT ACTUALLY DOES
<code>garak</code>	Adversarial / jailbreak probe suite against the target
<code>deepeval</code>	LLM-as-judge metrics (bias, toxicity, relevance, context-aware formulas)
<code>ragas</code>	Retrieval grounding — faithfulness, hallucination; runs in <code>.venv-ragas</code>
<code>presidio</code>	PII detection over target output
<code>pyrit</code>	Red-team orchestration; isolated <code>.venv-pyrit</code> (openai>=2.2 conflict)
<code>inspect_ai</code>	UK AISI Inspect evals
<code>langfuse</code>	Trace completeness (CM-039 only)
<code>vision</code> / <code>audio</code>	Multimodal generation probes; return <code>MediaAsset</code> s → <code>ExecutionArtifact</code>
<code>drift</code>	Black-box drift probing; aliased over the legacy <code>evidently</code> tool name
<code>threshold</code>	Derives a conservative score from the metric's own threshold
<code>mock</code>	Fixed <code>mock_score</code> ; only when explicitly requested

NO SYNTHETIC FAILURES

A metric whose tool has no integration (`promptfoo` , three of the four `langfuse` metrics) is returned `skipped` with an explicit reason. It used to fall back to the threshold evaluator, which derived a score just below the threshold — fabricating a guaranteed failure with no evidence behind it.

WHAT THE ORCHESTRATOR RECORDS AFTERWARDS

```
orchestration.py :: _record_pipeline_step(phase=metric_execution,
    entry_type="metric_execution_completed", event_type="metric_execution.completed",
    actor_type=tool, actor_id=payload.evaluator_name,
    payload={evaluator_name, evidence_created, metric_results_created,
        metric_result_ids=[...sorted...],
        content_digest=content_integrity.metric_result_content_digest(results)})
    # the digest lets a later verify pass detect that THESE rows were altered
    # after the fact – something the ledger's own hash chain cannot catch
```

07 Layer 3 — Specialist agents

Seven model-backed agents run in two stages: six fan out concurrently, then `risk_scorer` runs after a barrier so it sees a complete peer finding set. Each agent probes the live target, optionally calls a real evidence tool, reasons over both with the governance model, and returns findings.

Phase setup

```
services/specialist_agents/agent_execution.py :: run_agents(session, run_id, payload,
                                                         evaluation_plan, probe_budget_override, is_remediation_call)
├ run.status = agents_running; phase = specialist_agents → commit # skipped for remediation
├ gateway.start_log_capture(run_id, "specialist_agents")
├ probe_budgets = {item.agent_name: item.probe_budget for item in plan.activated_agents}
├ # or {name: 8} for a council re_probe (RE_PROBE_BUDGET_CAP)
├ _load_worker_snapshot(ai_system_id, run_id) # throwaway session, then detached:
├ ai_system · context_profile · capabilities · evidence · metric_results ·
├ metric_plan_items · existing_findings + register_capability_modalities()
├ _get_prior_metric_scores() # metric_id → score from the last completed run (drift baseline)
├ target_client = get_target_model_client_for_system(snapshot.ai_system)
├ # per-SYSTEM, not the global TARGET_* pair — that global resolver once sent a
├ # RAG chatbot audit's probes at the HR gateway
├ already_done = AgentExecution WHERE status=completed AND not is_remediation # resume
├ agents = registry.py :: select_agents(names, target_client, governance_client)
├ quality_agent · bias_agent · misuse_agent · drift_agent ·
├ compliance_mapper · risk_scorer · explainability_agent
├ # unknown name → 422 VALIDATION_ERROR {unknown_agents}
├ fanout = [a for a in agents if not a.aggregates_peer_findings]
├ aggregate = [a for a in agents if a.aggregates_peer_findings] # risk_scorer only
├ for each agent: AgentExecution(status=running, started_at, metadata_json={
├     execution_mode, execution_stage: parallel|aggregate, selected_by_request,
├     is_remediation, inputs: _agent_inputs(...)) → commit
├     # rows exist and are queryable WHILE the phase runs; inputs are written before
├     # the agent runs, so an agent that fails with zero findings still leaves a record
├     # of what it had been asked to check
```

metadata_json["inputs"] — what an agent was handed

`_agent_inputs()` records, before the agent runs: `dimension`, `assigned_metric_ids[]`, `assigned_metrics[]` (id + human name + dimension, taken from the run's own metric plan so labels can't drift from the catalog), `probe_budget`, `priority`, `activation_rationale`, `audit_scope_capabilities`, `assigned_metric_states[]` (status/score/threshold/passed as seen — these drive the probe-or-skip decision), and the visible counts of metric results, evidence records, and prior findings.

Fan-out and finalize

```
pool = ThreadPoolExecutor(max_workers=agent_execution_max_workers()) # default 3
# MULTIPLIES with AGENT_PROBE_MAX_WORKERS (6): 3 agents x 6 probes = up to 18
# concurrent requests into the audited system
for future in as_completed(futures, timeout=agent_execution_budget_seconds()): # 900s
    └ _run_in_worker(agent) → own Session → _execute_agent(agent, context, capture_buffer)
        └ bind_log_capture(buffer, agent_name=agent.name)
        └ findings = list(agent.evaluate(context)) # ← the agent body, next section
        └ except → error={error_type, message}, findings=[] # never raises
        └ finally → set_current_agent(None) # so the council's calls aren't misattributed
    └ _AgentOutcome(started_at, completed_at, findings, error,
        probe_count = probe_counts[a] + tool_probe_counts[a],
        probes_planned= probe_plan_counts[a], # kept SEPARATE from sent
        probes_skipped, probes_failed)
except FuturesTimeout:
    for pending: add_done_callback(_record_orphaned_agent_completion(..., late_logs))
        # the abandoned agent keeps probing and keeps committing artifacts; the
        # callback ledgers "agent.orphaned_after_timeout" and recovers its late LLM
        # calls via _LateLogSink. Its FINDINGS are deliberately discarded – the
        # phase's content digest is already checkpointed.
pool.shutdown(wait=False)

for agent in fanout_agents: # registry order, NOT completion order
    _finalize(agent, outcome)
    └ execution.status = failed|completed; started_at/completed_at = the REAL window
    └ execution.metadata_json["outputs"] = _agent_outputs(outcome)
        {probes_sent, probes_planned, probes_not_sent, probes_skipped,
        probes_failed, finding_count, findings_by_severity, finding_titles, error}
    └ if error and no findings → helpers.agent_failed_finding()
        # a failed agent's silence otherwise read as "clean" to the council,
        # which counts findings
    └ Finding(run_id, **payload.model_dump()) for each emitted finding
    └ append_ledger_entry("agent.completed"|"agent.failed", actor_type=agent,
        actor_id=agent.name, payload={phase, status, execution_stage, probe_count,
        probes_skipped, finding_count, started_at, completed_at, duration_ms,
        is_remediation, error}, commit=False)
        # per-agent windows are what let a reviewer confirm the fan-out actually
        # overlapped rather than take the stage label on faith
    └ session.commit() # per agent, not per phase

— barrier — session.flush()
for agent in aggregate_agents: # risk_scorer, over _list_findings(run_id)
    remaining = max(120.0, budget - elapsed) # _AGGREGATE_MIN_BUDGET_SECONDS
    # on its own thread so it is BOUNDED – it once blocked 7.8 HOURS inline
    # against a 900s phase budget on a throttled target
    _finalize(agent, outcome_or_TimeoutError)

_finalize_endpoint_coverage() # probes are now all logged with their endpoint:
probed = {e["endpoint_ref"] for e in buffer if e["call_type"]=="target"}
if len(registered) >= 2 and 0 < len(unprobed) < len(registered):
    Finding(**helpers.unprobed_endpoints_finding()) → commit
    # computed from the phase's own buffer, not from llm_call_logs – those rows
    # aren't written until the final commit
```

Inside one agent

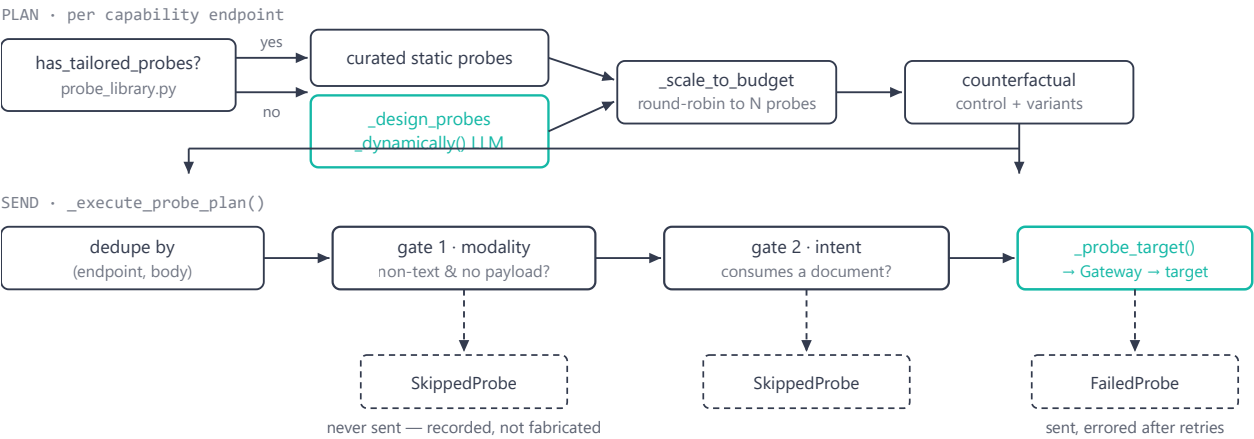
Every agent subclasses `ModelBackedAgent` (`services/agents/model_backed/base.py`). Taking `bias_agent.py` as the worked example:

```
BiasAuditorAgent.evaluate(context)  # name="bias_agent", probe_dimension="bias"
├─ _owned_metrics(metric_ids={CM-017...CM-021}, keywords=("fairness","bias",...))
│   if EMPTY → [helpers.coverage_gap_finding(reason="no_metrics_planned")]
│       # "never evaluated" must not be indistinguishable from "evaluated and clean"
├─ _metrics_for_review(owned) → (review, attention)
│   attention = failed U pending
│   review    = attention, OR all owned when _verify_even_when_passing()
│               # i.e. risk_tier == high – a high-risk system's passing fairness
│               # score must be VERIFIED with live probes, not trusted
│   both empty → _unprobed_dimension_findings() # records WHY nothing was probed
├─ probes = _run_probes(_PROBE_PROMPTS, context)      — see below
├─ tool_calls = _call_evidence_tool(tool_name="deepeval", metric_ids, context)
│   └─ for each matching plan_item: EVALUATORS["deepeval"].evaluate(...)
│       → ToolCallResult(tool_name, metric_id, formula, status,
│           normalized_score, passed, payload)
│   tool_probes = _count_target_calls(buffer) delta, filtered by agent_name
│       # filtering is essential, not defensive: agents share ONE buffer, and a
│       # plain delta once inflated misuse_agent to 53 calls when it made 27
│   → context.tool_probe_counts[name] += tool_probes
├─ _ask_governance_with_json_retry(task="bias_analysis", prompt=...) LLM
│   prompt carries: system name/type/risk_tier, metric statuses,
│                   probe_evidence = "\n\n".join(p.fenced ...),
│                   deepeval tool evidence lines
│   └─ _ask_governance() → Gateway → _parse_findings_json() (first [ ... last ])
│       └─ non-JSON → ONE retry with _JSON_RETRY_SUFFIX appended, then parse again
├─ parsed is not None → _findings_from_governance()
│   per item: Severity(item["severity"]) (fallback high), confidence,
│   dimension from the cited metric, evidence_ids = metric.evidence_ids
│   or the reviewed metrics' combined evidence, generated_by="governance_model"
└─ parsed is None → split_attention_metrics(attention)
    _deterministic_fallback(genuinely_failed) + metric_not_evaluated_finding(never_evaluated)
    # fallback findings ONLY for genuinely failed metrics – never fabricate a
    # failure from a passing metric under verification mode, and never report a
    # skipped/errored metric with the same "requires review" language as a failure
```

AGENT	PROBE_DIMENSION	EVIDENCE TOOL	LLM TASK LABEL
quality_agent	quality	ragas / deepeval	quality_evaluation
bias_agent	bias	deepeval	bias_analysis
misuse_agent	misuse	garak	misuse_detection
drift_agent	drift	drift	drift_analysis
compliance_mapper	transparency	presidio	transparency_analysis
explainability_agent	explainability	deepeval	explainability_analysis
risk_scorer	aggregate	—	oversight_analysis

`risk_scorer` is the only agent with `aggregates_peer_findings = True`. Its contract (`services/agents/risk_contract.py`) reads every finding accumulated in the run and emits a `risk_summary` finding carrying a `composite_score` — which the council's deterministic fallback later inverts into a confidence score.

The probe pipeline



failed > 0 and sent == 0 → AllProbesFailedError → agent failed → run degraded

Skipped and failed are kept strictly apart. A skip means the probe was never sent because the capability could not receive it; a failure means the target was asked and gave nothing back. Folding them together would let a dead endpoint read as a modality gap.

```

ModelBackedAgent._run_probes(fallback, context)
├ endpoints = context.probe_endpoints()
│   scoped → the selected capabilities' endpoint_refs
│   unscoped → EVERY registered capability's endpoint_ref
│
│   # collapsing to one endpoint meant image/video capabilities were
│   # never even attempted – not skipped-with-a-reason, never considered
├ profile = probe_library.classify_system(system_type)
├ per endpoint:
│   has_tailored_probes(dimension, endpoint_ref, profile)
│   └ yes → probes_for_endpoint() / _select_probes() → _scale_to_budget()
│       └ no → _design_probes_dynamically() LLM · dynamic_probe_design
│           template "orchestrator.probe_design" (content-addressed, hash-pinned)
│           → validate_dynamic_structured_probes(schema, ...) (non-text or doc-consuming)
│           → validate_dynamic_text_probes(...) (free-form text)
│           → None on any failure → caller falls back or records an honest skip
│   _with_structured_fallback() # a curated text plan that CANNOT be delivered to
│                               # this endpoint falls through to dynamic design; curated
│                               # bodies always win where they exist
│   _extend() → expand_counterfactual_probes()
│       # a fairness probe is only evidence if something can be compared:
│       # control + one variant per protected-attribute signal, identical otherwise
├ _execute_probe_plan(plan, agent_name, capabilities, dimension, dynamic_payloads)
│   └ dedupe on (endpoint actually called, body actually sent)
│       # curated payloads are pinned to their OWN endpoint, so fanning them across
│       # capabilities sent identical bodies repeatedly – one run's "221 probes"
│       # were 2 distinct questions asked ~110 times each
│   └ ThreadPoolExecutor(agent_probe_max_workers()=6).map(_send, plan)
│       _send(): gate 1 modality → gate 2 endpoint_answers_freeform_questions()
│           → payload_for_probe() / dynamic payload / synthesize_structured_body()
│           → _probe_target(), exceptions captured as FailedProbe
│       └ failed and not sent → raise AllProbesFailedError
├ context.probe_counts[name] = len(sent); probe_skips / probe_failures populated
└ for results with media: execution_artifacts.py :: record_execution_artifacts()
    → ExecutionArtifact(prompt_text, response_text, media_kind, mime_type,
                        data_base64, source_url) # failure here never fails a run

```

```

ModelBackedAgent._probe_target(endpoint_ref, prompt, capability_name, payload)
├ metadata = {"payload": payload} if payload else {}
├ response = self._target.invoke(TargetModelRequest(endpoint_ref, prompt,
│                               capability_name, metadata)) # ← the Gateway
├ sanitization.py :: sanitize_target_output(raw) → SanitizedTargetOutput(text, warnings)
├ sanitization.py :: fence_untrusted_target_output() # target output is NEVER mixed
│   # unfenced into a governance prompt
└ TargetProbeResult(probe_prompt, sanitized, fenced, latency_ms, trace_id,
                    endpoint_ref, capability_name, media[])

```

`RUN_AGENTS()` RETURNS

`AgentRunRead` — `agents_run[]` (`AgentRunSummary` : id, agent_name, finding_count, status), `executions[]`, `findings_created`, `findings[]`. Reused executions/findings from a prior partial attempt are folded in, so a resumed run's digest covers everything that exists, not just this call's delta.

`RUN.RESULT_SUMMARY` GAINS

`agents_run[]`, `agent_executions_failed`, `agent_findings_created`, `agents_reused_from_prior_attempt`, `agents_completed_at`, and `agent_execution_model` = `{parallel_stage[], aggregate_stage[], max_parallel_workers}` — so the execution model is stated from evidence.

08 Layer 4 — Deliberation council

Three agents in a bounded remediation loop: Synthesis → Devil's Advocate → Verdict, up to three iterations, with a deterministic router deciding each exit.
Forced dissent fires on *every* pass, not just the first.

```

services/deliberation_council/deliberation.py :: deliberate(session, run_id, payload)
└ existing Verdict for this run → raise ResourceConflictError # one verdict per run, ever
└ gateway.start_log_capture(run_id, "deliberation_council")
└ governance_client = _get_governance_client()
  └ on failure → MockGovernanceModelClient, logged at ERROR
    governance_is_mock = _is_mock_governance_client(client)
    # captured ONCE here so the verdict AND the run both carry it – a mock
    # fallback is otherwise indistinguishable from a real deliberation
└ registry = configs/prompt_registry.py :: PromptRegistry.from_directory()
  # templates are CONTENT-ADDRESSED: each agent pins (template_id, sha256), so a
  # shipped version is immutable and old runs keep resolving the one they used
└ SynthesisAgent · DevilsAdvocateAgent · VerdictAgent (all share that registry)
└ iteration = _read_iteration_counter(run) # result_summary["_council_iteration_count"]
  # survives crash-resume; never resets to 0
└ findings = _list_open_findings(); metric_results = _list_metric_results()
└ while True:
  iteration += 1
  current = _latest_findings_per_agent(findings) # latest-per-agent read rule
  probes = _get_real_probe_counts(session, run_id)
    # Σ AgentExecution.metadata_json["probe_count"] per agent – REAL telemetry,
    # re-read each iteration because a re_probe adds executions mid-loop

  └ synthesis_agent.synthesize(findings, metric_results, iteration, real_probe_counts)
    └ LLM · council_synthesis template synthesis_agent.council_memo (v6)
      → SynthesisMemo(narrative, risk_summary, dimensions[], sample_sizes{},
        conflicts[], claims[], unused_findings[], coverage)
        claims[] = provenance edges: each claim cites finding_ids + a role
          (primary / corroborating / compounding)
        _validate_provenance() drops citations the model invented;
        ProvenanceCoverage is reported even when INCOMPLETE
        sample_sizes is always overwritten with real telemetry after parsing

  └ da_agent.object_to(memo, current_findings)
    └ LLM · council_devils_advocate
      sees the raw findings too, not just the memo, so it can catch an OMISSION
      → [Objection(objection_id, target_agent, category, argument,
        suggested_fix, remediation_hint,
        attacks: evidence|inference, target_claim_id, target_finding_ids)]

  └ verdict_agent.adjudicate(memo, objections, findings, metrics, iteration)
    └ LLM · council_verdict threshold _SUFFICIENCY_THRESHOLD = 0.65
      └ _parse_verdict() → confidence_score - iteration_penalty → clamp
        score < 0.65 → sufficient forced False (threshold_guard)
        label ∉ {approved, conditional_approval, blocked} → "blocked"
      └ _validated(objections=...) # drop target_agent not in REGISTERED_AGENT_NAMES;
        # drop objection ids that don't exist this iteration
      └ _apply_policy_floor(metric_results)
        # ANY conclusively-failed metric → sufficient=False, label="blocked",
        # tier=human_review. Enforced in CODE on both paths: the LLM template
        # carries no failed-metric rule, so the same evidence used to yield
        # approved/autonomous via the judge and blocked/human_review via the
        # fallback, depending only on whether the judge was reachable.
      └ verdict.remediable = _remediation_can_change_outcome()
        └ no LLM → _deterministic_fallback()
          risk_summary finding present → confidence = 1 - composite_score,

```

```

        capped at 0.45 if composite ≥ 0.75 or a critical finding is open
        else 0.95 - 0.15·failed - 0.05·pending - Σ severity penalties
    every path builds a ConfidenceDerivation: source, raw_score,
    final_score, threshold, policy_floor_applied, sufficiency_reason,
    steps[] - mechanical, recomputable, never narrated by the model

└ _persist_iteration_counter()    # BEFORE routing, so a crash here can't reset it
└ _append_council_iteration_state(entry_type="council_iteration")
    one append-only state entry per iteration: synthesis (+claims, coverage),
    objections (+attacks), verdict (+derivation steps)
└ session.commit(); session.refresh(run)    # per iteration, so pollers see progress
└ decision = remediation_router.py :: route(verdict_out, iteration)

```

The router's four exits

ORDER	CONDITION	EXIT	EFFECT
1	sufficient	action	Break; proceed to Layer 5 at the verdict's own tier
1b	not remediable	action	Insufficient but <i>conclusive</i> — a failed metric can't be cleared by any remediation path, so decide now
2	iteration ≥ 3	exhausted	Force blocked / human_review + build_exhaustion_memo()
3	otherwise	remediate	re_deliberate · re_probe · re_plan

WHY EXIT 1B EXISTS

No remediation path re-runs metric execution, so once a metric has conclusively failed the sufficiency gate (`failed == 0`) can never be satisfied. The loop used to run to its cap anyway and stamp the verdict with a memo reading "unresolved uncertainty: the Council lacked enough evidence to decide" — the opposite of the truth. Exit 1b decides in one iteration and says so positively: *DECIDED ON CONCLUSIVE EVIDENCE*. An `error` metric stays remediable, because there the evidence is *missing*, not negative.

REMEDIATION

```
_apply_remediation(remediation_type, target_agent, objections, objections_upheld)
├ re_plan → _resolve_re_plan_target()
│   match upheld objection text against DIMENSION_TO_AGENT_NAME, excluding agents
│   that already completed this run → activate a DORMANT specialist, or no-op
├ re_probe / re_plan with a target:
│   parent_buffer = get_log_buffer()          # run_agents() does its own start/drain
│   run_agents(agent_names=[target], probe_budget_override=8, is_remediation_call=True)
│   ledger "remediation.re_plan_activated" | "remediation.{type}_failed"
│   finally: bind_log_capture(parent_buffer)
│
│   # without the restore, every council LLM call AFTER this point vanished –
│   # _append_log() no-ops when the buffer is None
└ re-read open findings + metric results → next iteration
```

PERSISTING THE VERDICT

```
_persist_verdict() → Verdict(run_id UNIQUE, confidence_score, action_tier, label,
                               synthesis=memo.narrative, objections[], reasoning,
                               required_actions[])
├ required_actions: exhaustion → one human_reviewer action;
│   otherwise one per open finding's recommended_action
│   # deterministic, derived from findings – not from the LLM's verdict text
└ reasoning gains a bracketed audit tail:
    [Council iterations: N | iteration_penalty applied: ... | LOOP EXHAUSTION: ... |
    DECIDED ON CONCLUSIVE EVIDENCE: ... | NOT EVIDENTIAL: no live governance model
    was available, so this verdict was deliberated by the MOCK council client ...]
    # stated on the verdict itself, not only in result_summary – the reasoning text
    # is what gets read, quoted, and exported into reports

_update_run() → result_summary += {council_label, council_confidence_score,
    council_deliberated_at, council_iterations, council_exhausted,
    council_governance_is_mock}

_append_council_ledger("council_deliberation.completed") # payload.phase set so the
    # Runtime Event Stream renders it on the Council layer
for entry in drain_log_capture(): session.add(LLMCallLog(...)) → commit
```

RETURNS

```
CouncilDeliberationRead — verdict,
finding_count, open_finding_count,
metric_result_count, failed_metric_count,
pending_metric_count, highest_severity,
created_verdict.
```

ACTION TIERS

```
approved → autonomous, conditional_approval →
supervised, blocked → human_review
(_safe_action_tier()); unknown labels map to
human_review).
```

WHAT THE PIPELINE DOES WITH A COUNCIL FAILURE

```
# orchestration.py :: _execute_and_report()
try:    council_result = council.deliberate(...)
except ResourceConflictError:                # a Verdict already exists
    if not verdicts.py :: council_produced_verdict(session, run_id):
        raise ApplicationError(409, "VERDICT_NOT_COUNCIL_PRODUCED")
        # a verdict written through the public route before the run reached the council
        # used to be adopted as the council's own outcome and exported as though the
        # council had reasoned its way to it. The LEDGER is the provenance record.
    council_result = council.get_existing_deliberation() # created_verdict=False
except Exception as exc:
    council_error = {error_type, message}
    session.rollback() # deliberate() may have left the session dirty; earlier
                      # phases already committed, so only council partials are lost
    _record_pipeline_step(event_type="council_deliberation.failed")
    # the run continues → report + terminal status, degraded with council_failure
```

09 Layer 5 — Action & reporting

```
# phase moves first, so the Action & Reporting stage actually activates in the UI;
# status stays non-terminal (report_ready would stop the frontend poll early)
run.current_phase = action_reporting → commit

services/action_reporting/reports.py :: build_governance_report(session, run_id)
├ run · ai_system # missing AISystem → 404 ResourceNotFoundError
├ ApplicationContextProfile · AISystemCapability[] (created_at ASC)
├ EvidenceRecord[] · MetricResult[] · AgentExecution[] · Finding[] (created_at ASC)
├ Verdict # .first() – a MISSING verdict does not prevent a report; that is the
  # whole point of degrading rather than failing. The audit record survives.
├ governance_state.py :: verify_state_chain(run_id)
├ audit_ledger.py :: verify_ledger_chain(run_id)
├ content_integrity.py :: verify_content_integrity(run_id)
  → AuditLedgerChainVerification(valid, entry_count, failed_entry_id, reason,
  content_valid, content_checks)
├ build_metric_plan() # recomputed, so the report states its own plan
└ endpoint_coverage.py :: build_endpoint_coverage(run_id)
  per endpoint: probes_sent · probes_failed · probes_skipped · requests_made ·
  agents{} · error_types{} · sample_error
  # a probe is one logical question (one call-log row); a REQUEST is one HTTP
  # round trip. The Gateway retries, so a throttled probe can be 3 requests.
  # Both are reported, separately and labelled.
```

RETURNS GOVERNANCEREPORTREAD

```
run, ai_system, context_profile, capabilities[],
metric_plan, agent_executions[], evidence[],
metric_results[], findings[], verdict,
state_chain, ledger_chain, endpoint_coverage,
counts.
```

COUNTS

```
capabilities, planned_metrics, planned_controls,
evidence, metric_results, agent_executions,
findings, state_entries, endpoints_registered,
endpoints_unprobed.
```

The report is read-only and idempotent — `GET /evaluation-runs/{id}/report` recomputes it on demand, including both chain verifications. A sibling route, `GET /{id}/framework-map` (`action_reporting/framework_maps.py`), projects the same evidence onto per-framework control coverage.

10 Finalize — how a run reaches a terminal status

```
# tail of _execute_and_report()
failed_executions = [e for e in agent_result.executions if e.status == failed]
degraded = {}
└ failed_executions → degraded["specialist_agent_failure"] = {message, failed_agents[]}
└ council_error      → degraded["council_failure"]           = {message, error}
  |   "...this run carries NO governance decision. The metric results and specialist
  |   findings it did collect are preserved and reported; re-run the council..."
└ report_error       → degraded["report_failure"]            = {message, error}

# collected TOGETHER, so a run that lost an agent AND the verdict reports both –
# dropping one would understate how incomplete the audit is
run.status = degraded if degraded else completed
run.error_summary = {degraded_reason: "a,b,c", message: "<flat one-liner>", **degraded}
run.current_phase = completed; completed_at = completed_at or now → commit

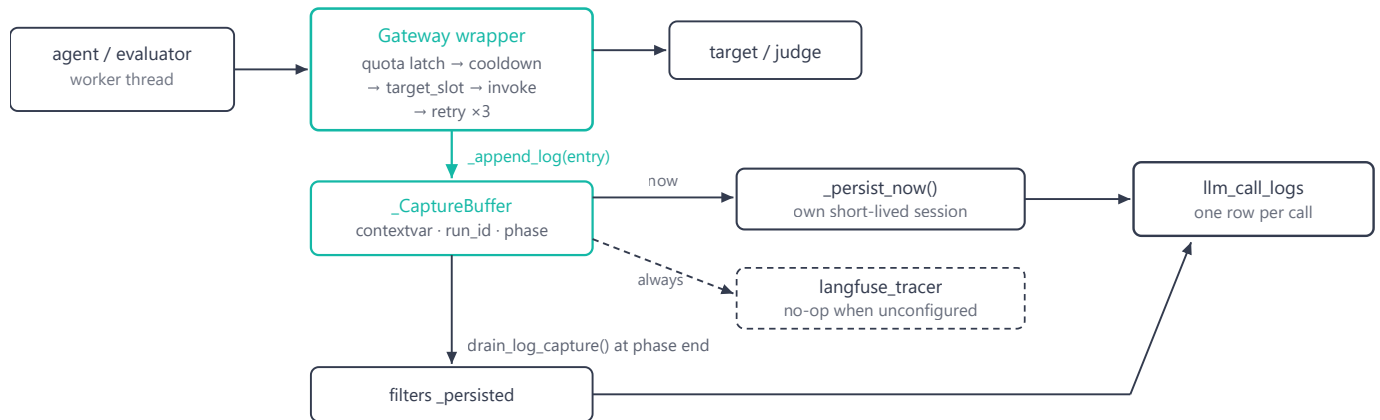
→ GovernancePipelineRunRead(run_id, context_assembly, evaluation_plan,
                             metric_execution, agent_run, council, report)
```

```
orchestration.py :: _finalize_run(run_id, status, error=None) # own session
└ run.status in {completed, failed, cancelled} → return # already terminal
└ run.status == degraded and status != failed → return
  |   # a success finalize must NOT overwrite degraded – the pipeline decided the run
  |   # was degraded and that decision stands. A FAILURE finalize must be able to:
  |   # a run that degraded in the agent phase and then genuinely crashed in the
  |   # council was otherwise left at "degraded" with no error_summary and no
  |   # completed_at, and – being absent from _INTERRUPTED_STATUSES – was never
  |   # revisited by startup reconciliation either. The misreport was permanent.
└ status == completed → current_phase = completed
└ error → error_summary MERGED (not replaced) with {error_type, error_code, message}
  |   # error_code = ApplicationError.code – without it the only record is prose,
  |   # which callers end up string-matching on
└ completed_at = completed_at or now → commit
└ try: append_ledger_entry(event_type="run.finalized",
                           payload={status, error_type, message})
    except: log only # a ledger failure must never block finalization
```

. LLM gateway & call logs

Every model call in the platform — target probes and governance reasoning alike — passes through `services/model_clients/gateway.py`. It is the only place that retries, throttles, and records.

`bind_log_capture(buffer, agent_name)` — contextvars do not cross thread boundaries



Two write paths to one table. `_persist_now()` makes each call queryable the instant it finishes; the phase-end drain writes only what the immediate path missed, so the log can only ever be *more* complete, never double-inserted.

Retry, throttle, and quota

```
GatewayTargetModelClient.invoke(request)
for attempt in range(3):
    | _check_quota(endpoint)      # latched TargetQuotaExhausted → raise, NO request sent
    | _await_cooldown(endpoint)  # shared instance-level cooldown, before taking a slot
    | with target_throttle.target_slot(target_modality.modality_for(endpoint)):
    |     response = self._inner.invoke(request)
    |     # ONE process-wide slot per ATTEMPT, around the network call only, so a
    |     # backoff sleep never occupies a slot. Video gets its own cap (default 1).
    | _append_log({task=_probe_name(capability_name), call_type="target", endpoint_ref,
    |             attempts, model, deployment_name, client_mode, routed_via, token counts,
    |             estimated_cost_usd, latency_ms, status="success", request/response_chars,
    |             trace_id, policy_flags, prompt_text, response_text})
    | except exc:
    |     _is_retryable(exc):
    |         TargetQuotaExhausted          → False    # retrying spends a refused allowance
    |         exc.terminal_refusal          → False    # moderation/content-policy block arrives
    |                                             # as the target's own 502; deterministic
    |
    |         RateLimitError|APIConnection|APITimeout → True
    |         HTTPError.code ∈ {408,425,429,500,502,503,504} → True
    |         URLError | TimeoutError        → True
    |     _is_rate_limit(exc): 429, OR a 429 tunnelled inside the target's own 502
    |         # "Sora create failed [429]: Too many running tasks" – keying only on
    |         # exc.code read that as a server error, so no cooldown was published and
    |         # every worker kept firing into the same limit
    |     | _quota_exhaustion(exc): Retry-After > 300s → _enter_quota_exhausted(), break
    |     | else _enter_cooldown(retry_after or 2**attempt) # EXTENDS, never shortens
    |     | non-retryable → _append_log(status="error"|"rate_limited", attempts=0 if
    |                               quota-latched else attempt+1, error_type, error_detail,
    |                               prompt_text kept, response_text=None) → raise
```

GatewayGovernanceModelClient.complete() mirrors this without the throttle/cooldown, logging `call_type: "governance"` and keeping `prompt_text` even when the call fails — a failed governance call is exactly when a reviewer needs to see what was asked.

llm_call_logs — the row

COLUMN	MEANING
run_id , agent_name , phase	Who and which layer. phase is stamped from the buffer, not per call site, so nothing can forget it
task	The probe name (endpoint tag stripped) or the governance task label; NOT NULL , coalesced to "target_probe"
call_type	target Or governance
endpoint_ref	Which audited surface (target calls only) — indexed, because every per-endpoint rollup groups on it
attempts	Physical HTTP requests this one logical call cost
model , deployment_name , client_mode , routed_via	live vs mock ; LiteLLM proxy vs direct
prompt_tokens ... estimated_cost_usd	Cost via _estimate_cost_usd() against the _COST_TABLE
latency_ms , status	Measured, not assumed. success / rate_limited / error
error_type , error_detail	error_detail carries the target's own response body — where an audited system explains itself behind a 502
prompt_text , response_text	Full transcript, both call types, kept on failures too

```
GET /api/v1/evaluation-runs/{run_id}/llm-calls
└─ services/llm_gateway/call_log.py :: get_llm_call_log_summary()
  └─ SELECT LLMCallLog WHERE run_id ORDER BY created_at ASC
    → LLMCallLogSummary(call_count, total_prompt_tokens, total_completion_tokens,
      total_tokens, estimated_total_cost_usd, live_call_count, mock_call_count,
      error_count, calls[])
    # frontend: CallTranscripts.tsx renders these per agent
```

Client resolution

```
model_clients/registry.py
get_target_model_client_for_system(ai_system) # per-SYSTEM — always prefer this
└─ _infer_target_kind(): metadata catalog_url | x-api-key | "/api/v1/ai" → hr_gateway
  | x-functions-key | rag/chat system_type → techvest
└─ same host as the configured HR gateway → hr_gateway
└─ hr_gateway → HRGatewayTargetModelClient
└─ techvest → TechVestTargetModelClient
└─ any registered http(s) endpoint → GenericHTTPTargetModelClient.for_system()
└─ else → get_target_model_client(): TARGET_* → LiteLLM → Azure OpenAI → Mock

get_governance_model_client(): LiteLLM proxy → Azure OpenAI → Mock
# every branch returns a Gateway-wrapped client, which is why _probe_target()
# does not acquire a throttle slot itself
```

Integrity — two hash chains and a content digest

STATE CHAIN

`governance_state_entries` — one entry per phase artifact. `append_state_entry()` hashes `{run_id, sequence_number, entry_type, source, phase, payload, previous_hash}` as canonical JSON (sorted keys, compact separators). `UNIQUE(run_id, sequence_number)`.

AUDIT LEDGER

`audit_ledger_entries` — one entry per governance event. `append_ledger_entry()` hashes `{run_id, event_type, actor_type, actor_id, payload, previous_hash}`. `commit=False` folds an entry into a caller's transaction.

CONTENT DIGEST

`services/content_integrity.py` — SHA-256 over the exact `Finding` / `MetricResult` rows a ledger entry summarized, at write time. The chains prove the *ledger* rows weren't altered; the digest catches changes to the ordinary mutable tables they describe.

Both verifiers walk the whole chain unpaginated and return the first breach: `sequence_gap`, `previous_hash_mismatch`, or `entry_hash_mismatch`. There is a matching invariant suite in `services/audit_invariants.py`.

EVERY LEDGER EVENT A FULL RUN CAN EMIT

EVENT_TYPE	ACTOR_TYPE / ACTOR_ID	WRITTEN BY
context_assembly.completed	system / context_assembly	assembler.py
evaluation_plan.prepared	system / adaptive_orchestrator	planner.py
plan.awaiting_approval	system / orchestrator	_pause_for_approval()
plan.approved	user / approver	approve_plan()
metric_execution.completed	tool / evaluator_name	_record_pipeline_step()
metric.orphaned_after_timeout	system / metric_execution	straggler callback
agent.completed · agent.failed	agent / agent_name	_finalize(), one per agent
agent.orphaned_after_timeout	system / agent_execution	straggler callback
agent_execution.completed	agent / agent_orchestrator	_record_pipeline_step() + digest
council_deliberation.completed	system / deliberation_council	_append_council_ledger()
council_deliberation.failed	system / council_service	_execute_and_report()
remediation.re_plan_activated	system / deliberation_council	_apply_remediation()
remediation.{type}_failed	system / deliberation_council	_apply_remediation()
governance_report.generated	system / report_service	_record_pipeline_step()
run.finalized	system / orchestrator	_finalize_run()
run.reconcile_failed	system / orchestrator	reconcile_interrupted_runs()

Every payload carries `phase` — the Live Run view's Runtime Event Stream filters on it, so an event without it exists but renders on no layer.

Failure, degradation, and resume

THE DEGRADATION LADDER

WHAT FAILS	CONSEQUENCE	RECORDED AS
One probe	Sibling probes still land	<code>FailedProbe</code> in <code>metadata_json.probes_failed</code>
All of an agent's probes	Agent fails	<code>AllProbesFailedError</code> → <code>agent.failed</code> + synthetic coverage finding
One metric	Others persist	<code>MetricResult.status = error</code> , run <code>degraded</code>
One agent	Phase continues	<code>AgentExecution.status = failed</code> , run <code>degraded</code>
Metric phase budget	Unfinished metrics skipped	<code>metric.orphaned_after_timeout</code>
Agent phase budget	Stragglers abandoned, findings discarded	<code>agent.orphaned_after_timeout</code> + late-log recovery
Council	Report still built, no verdict	<code>council_failure</code> in <code>error_summary</code>
Report	Run still reaches terminal	<code>report_failure</code> in <code>error_summary</code>
Anything unhandled	Run marked failed	<code>_finalize_run(failed)</code> + <code>run.finalized</code>

STARTUP RECONCILIATION

```
main.py @app.on_event("startup") → daemon thread → reconcile_interrupted_runs(session)
# On a THREAD because reconciliation RESUMES a run – real probes, minutes of work.
# Inline it sat in front of the socket bind ("Waiting for application startup",
# port never opened), and a mid-resume death faced the next boot with the same
# unfinished run – an unbounded restart loop that never served a request.

SELECT EvaluationRun WHERE status IN (_INTERRUPTED_STATUSES + degraded)
  _INTERRUPTED_STATUSES = created · context_assembly · planned ·
                        metrics_running · agents_running · council_running
for run in stuck:
  | degraded and _is_interrupted_degraded(run) is False → skip
  |   # degraded + current_phase == completed means the run genuinely finished;
  |   # the phase is what tells the two kinds of degraded apart
  | is_awaiting_plan_approval(run) → skip   # waiting on a person, not a crash
  | a Verdict exists → the council was reached → mark completed (or keep degraded),
  |   phase=completed, result_summary["reconciled"]
  | status ∈ {created, context_assembly} → run_governance_pipeline() from scratch
  |   # no plan was ever persisted; context assembly makes no target calls, so
  |   # retrying it has no side effects
  | otherwise → resume_governance_pipeline()
  |   # run_metrics skips already passed/failed metrics; run_agents skips
  |   # already-completed non-remediation agents – no probe is repeated
  | resume itself raised → status=failed, error_summary["context"] = "Resume attempt
    after worker restart failed." + ledger "run.reconcile_failed"
```

Read-back API surface

What the frontend calls while a run is in flight and after it lands. All GET, all unauthenticated by design (the auth middleware protects mutations only, so SSE and `<img>` sources work without headers).

ROUTE	SERVICE	RETURNS
GET /evaluation-runs/{id}	<code>get_evaluation_run</code>	Status, phase, <code>result_summary</code> , <code>error_summary</code> , <code>plan_approved_at</code>
GET /{id}/progress/stream	SSE, 2 s tick	<code>{status, current_phase, progress %, agents[], probe_count, finding_count, result_summary}</code> ; closes on a terminal status
GET /{id}/context-assembly	<code>get_latest_context</code>	<code>ContextAssemblyRead</code> from the state chain
GET /{id}/evaluation-plan	<code>get_latest_plan</code>	<code>EvaluationPlanRead</code> from the state chain
GET /{id}/metric-plan	<code>build_metric_plan</code>	<code>MetricPlanRead</code> , recomputed — the approval modal's source
GET /{id}/agents/executions	<code>list_agent_executions</code>	Rows with <code>metadata_json.inputs</code> / <code>.outputs</code>
GET /{id}/findings	<code>routes/findings.py</code>	Findings with severity, confidence, <code>evidence_ids</code>
GET /{id}/evidence	<code>routes/evidence.py</code>	<code>EvidenceRecord</code> s + raw tool payloads
GET /{id}/llm-calls	<code>get_llm_call_log_summary</code>	Token/cost totals + every transcript
GET /{id}/endpoint-coverage	<code>build_endpoint_coverage</code>	Per-endpoint probes sent / failed / skipped / requests
GET /{id}/execution-artifacts	<code>list_execution_artifacts</code>	Metadata only (<code>has_media</code>), so the list stays light
GET /{id}/execution-artifacts/{aid}/media	base64 → bytes	Raw bytes with the real <code>Content-Type</code>
GET /{id}/ledger · /state	<code>audit_ledger.py</code> · <code>governance_state.py</code>	Paginated chains, filterable by event/actor/phase/source
GET /{id}/report	<code>build_governance_report</code>	The full package + both chain verifications
GET /{id}/framework-map	<code>build_framework_compliance_map</code>	Evidence projected onto framework controls

ONE SSE SUBTLETY

Each 2-second tick opens its *own* short-lived session. Looping on the request-scoped session made its identity map keep returning the first snapshot of the run row — status and phase never advanced, the progress bar froze at whatever phase the stream connected during, a terminal status was never observed, and a pool connection stayed pinned for the run's lifetime.

Frontend consumers: `hooks/useEvaluationRunner.ts` (create → orchestrate → poll), `hooks/useRunProgress.ts` (SSE), `components/execution/*` (one panel per layer, `CallTranscripts.tsx` for LLM logs, `MetricPlanApprovalModal.tsx` for the gate, `RuntimeEventStream.tsx` for the ledger, `CouncilProvenance.tsx` for claims and objections).

Everything one run writes

TABLE	ROWS PER RUN	WRITTEN BY
<code>evaluation_runs</code>	1, mutated through every phase	All phases
<code>governance_state_entries</code>	~5 + 1 per council iteration	<code>append_state_entry()</code>
<code>audit_ledger_entries</code>	~8 + 1 per agent + remediation/orphan events	<code>append_ledger_entry()</code>
<code>evidence_records</code>	1 per evaluated metric	<code>run_metrics()</code>
<code>metric_results</code>	1 per evaluated metric	<code>run_metrics()</code>
<code>agent_executions</code>	1 per agent (+1 per remediation call)	<code>run_agents()</code>
<code>findings</code>	N per agent + coverage-gap findings	<code>_finalize()</code> , <code>_finalize_endpoint_coverage()</code>
<code>verdicts</code>	Exactly 1 (<code>UNIQUE run_id</code>), or 0 if the council failed	<code>_persist_verdict()</code>
<code>llm_call_logs</code>	1 per model call, all four LLM-bearing phases	Gateway <code>_append_log()</code>
<code>execution_artifacts</code>	1 per probe response carrying media	<code>record_execution_artifacts()</code>

TUNING KNOBS

All resolve through `services/concurrency_settings.py` at call time — never `os.getenv` at import. The app reads `<repo>/ .env` via `pydantic-settings`, which never populates `os.environ`, so the documented `.env` overrides used to do nothing at all.

SETTING	DEFAULT	BOUNDS
METRIC_EXECUTION_MAX_WORKERS	3	Concurrent metric evaluations. Was 6 until the shared judge returned a 429 storm
METRIC_EXECUTION_BUDGET_SECONDS	600	Scaled by metric count; unfinished metrics → <code>skipped</code>
AGENT_EXECUTION_MAX_WORKERS	3	Concurrent agents. <i>Multiplies</i> with the probe width
AGENT_EXECUTION_BUDGET_SECONDS	900	Shared by fan-out and aggregate; aggregate floor 120 s
AGENT_PROBE_MAX_WORKERS	6	Concurrent probes within one agent
AGENT_TARGET_MAX_INFLIGHT	= probe width	Process-wide ceiling on requests into the target
AGENT_TARGET_MAX_INFLIGHT_VIDEO	1	Providers limit simultaneously <i>running</i> render jobs
TARGET_QUOTA_EXHAUSTED_SECONDS	300	A named wait above this is exhaustion, not throttling
LLM_CALL_TIMEOUT_SECONDS	60	Per call; video generation gets 600
ADAPTIVE_PLAN_REVIEW_ENABLED	true	Off = evaluate the full applicable catalog

TEN LLM CALL SITES IN ONE FULL RUN

1. `evaluation_plan_review` — planner narrows the metric set
2. `dynamic_probe_design` — per capability with no tailored probe (0..N)
3. Evidence-tool judge calls inside `deepeval` / `ragas` / `garak`
4. –8. One reasoning call per specialist agent (`bias_analysis` , `misuse_detection` , `quality_evaluation` , `drift_analysis` , `transparency_analysis` , `explainability_analysis` , `oversight_analysis`) — doubled on a non-JSON retry
5. `council_synthesis` + `council_devils_advocate` + `council_verdict` , once per iteration (×1–3)
6. Plus every target probe, logged as `call_type: "target"`

Traced from backend/app and frontend/src on the feature/p2-run-pipeline-fixes-and-ui branch. Companion docs: docs/SPEC.md, docs/ARCHITECTURE.md, docs/DATA_MODEL.md, docs/API_CONTRACTS.md, docs/RUNBOOK.md.